

ARRAY

Arrays

- Idea: fornire il concetto di array presente nei più comuni linguaggi di programmazione
- Un array è una struttura statica, una volta creato la sua dimensione (numero di elementi che lo compongono) non può essere più modificata
- Il principale vantaggio dell'array è l'accesso diretto a ciascuno elemento in un tempo $O(1)$.
- In generale gli arrays sono omogenei, cioè ogni elemento è dello stesso tipo. Questo limite può essere superato con il polimorfismo.

Arrays

- ▣ L'array è un oggetto.
- ▣ L'array può contenere elementi che sono:
 - ▣ tipi *primitivi*
 - ▣ *oggetti* (in realtà riferimenti ad oggetti!!!).

Arrays: dichiarazione

- Gli array Java sono oggetti, istanze di una classe speciale denotata da `[ ]`
- La posizione delle `[ ]` è a scelta: o dopo il nome, come in C, oppure di seguito al tipo (novità)

Dichiarazione di array Java:

```
<elemType>[ ] <arrID>;  
           oppure  
<elemType> <arrID>[ ];
```

**La dimensione si
specifica all'atto
della creazione**

```
int[ ] gradi;  
int gradi[];  
float pressione[];  
boolean[] stato;
```

Arrays: istanziazione

▣ Istanziione array Java:

```
<arrID> = new <elemType>[<size>];
```

▣ Esempi:

```
int gradi[];  
float pressione[];  
boolean[] stato;
```



```
gradi = new int[10];  
pressione = new float[100];  
stato = new boolean[15]
```

Ogni elemento dell'array è una variabile, se gli elementi dell'array sono di un tipo primitivo (int, float, char, ...)

```
int v[]
```

```
v = new int[3];
```



Arrays dichiarazione e istanziazione e inizializzazione

■ Dichiarazione e istanziazione di array Java :

```
<elemType>[ ] <arrID> = new <elemType>[<size>];
```

oppure

```
<elemType> <arrID>[ ] = new <elemType>[<size>];
```

■ Esempio

```
int[ ] gradi = new int[10];
```

oppure

```
int gradi[] = new int[10];
```

Java example:

```
int[] x = {10, 100, 90, 50, 45};
```

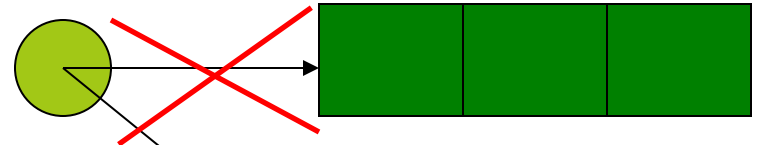
```
int x[] = {10, 100, 90, 50, 45};
```

Errori nella gestione degli array

- Comprendere la differenza fra riferimento ad array (array id) e l'oggetto array

- Esempio:

```
int oldcount[] = new int[3];
```



```
int counts[] = new int[3];
```



```
oldCounts = counts;
```

Errori nella gestione degli array

```
float[] pressione = new float[3];
```

```
int i=3;
```

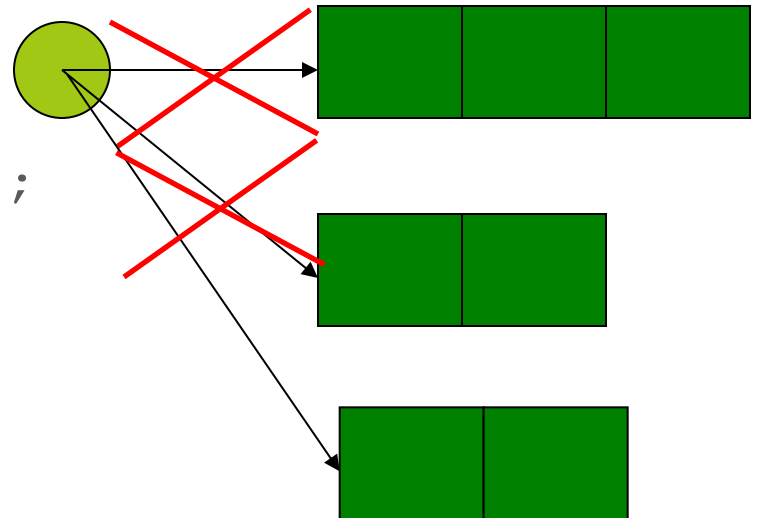
```
int j=1;
```

```
while(i>j) {
```

```
    pressione = new float[2];
```

```
    j++
```

```
}
```



Arrays

```
int[ ] gradi = new int[10];  
int i; for (i=0; i < gradi.length; i++) {  
    gradi[i] = 0;  
} // for loop
```

Se si cambia la dimensione dell' array, questo ciclo non deve essere modificato

Notes:

- per ogni Array e' conosciuta la dimensione
- *length* e' un attributo non un metodo
- gli Arrays sono dimensionati staticamente: la lunghezza non puo' essere modificata dopo l' istanziamento.
- tutti gli arrays sono oggetti che devono essere dichiarati, istanziati e inizializzati

Arrays

Notes:

- Indice dell' array inizia da 0 non da 1
- *length* e' uguale all'ultimo indice + 1

```
int[ ] gradi = new int[10];  
int i;  
for (i=0; i< gradi.length; i++) {  
    gradi[i] = 0;  
} // for loop
```

Dov'è l'errore?

- Il codice tenta di accedere agli elementi 1..10
- ma Dovrebbe accedere agli elementi 0..9

Problema

Dato un array di int, ritorna l'elemento maggiore.

```
public int maggiore(int[ ] myArray) {  
    int massimo = -9999;  
    int i;  
    for (i = 0; i < myArray.length; i++)  
    {  
        if ( massimo < myArray[i])  
            massimo = myArray[i];  
    }  
    return massimo;  
}
```

Cosa accade se
l'array contiene solo
Valori minori di -9999?

Problema

```
public int maggiore(int[] myArray) {  
    int massimo = myArray[0];  
    int i;  
    for (i = 1; i < myArray.length; i++)  
    {  
        if (massimo < myArray[i])  
            massimo = myArray[i];  
    }  
    return massimo;  
}
```

Esempio

- Dato un array di int, ritorna l'indice dell'elemento maggiore.
- Scrivere un metodo Java che accetta un array di int e ritorna un array in cui ciascuno degli elementi e' il doppio del valore originale.

Arrays

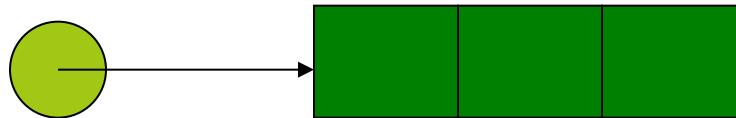
- Se si ha un *array di oggetti*, allora:
 - l'identificatore dell'array è un riferimento ad un array di oggetti
 - ogni elemento dell'array è un riferimento a un oggetto della classe specificata come tipo base dell'array
- Istanziare l'array di oggetti non implica l'istanziamento dei vari oggetti che costituiscono gli elementi dell'array.
- ***Gli elementi devono essere esplicitamente istanziati.***

Array in Java

ATTENZIONE

ogni elemento dell'array: è una variabile, se gli elementi dell'array sono di un tipo primitivo (`int`, `float`, `char`, ...)

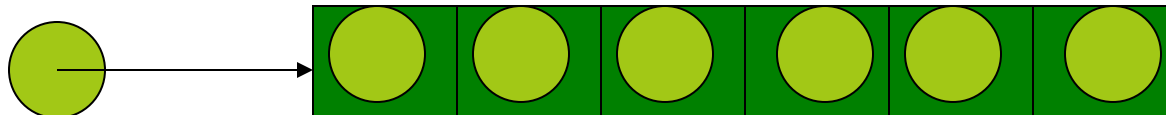
```
v = new int[3];
```



è un riferimento a un (futuro) oggetto, se gli elementi dell'array sono (riferimenti a) oggetti

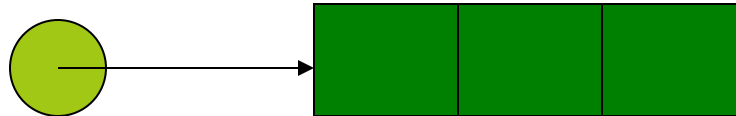
```
w = new Counter[6];
```

Inizialmente tutti `null`

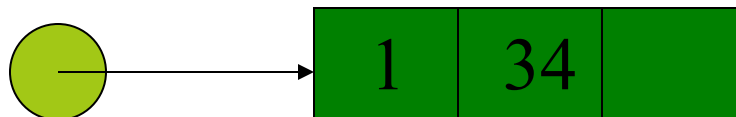


Array in Java

Nel primo caso ogni elemento dell'array è una normale variabile usabile così com'è



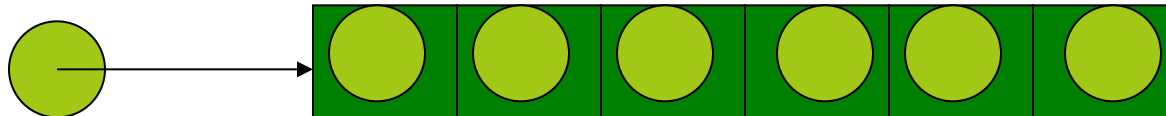
```
v = new int[3];  
v[0] = 1; v[1] = 34;
```



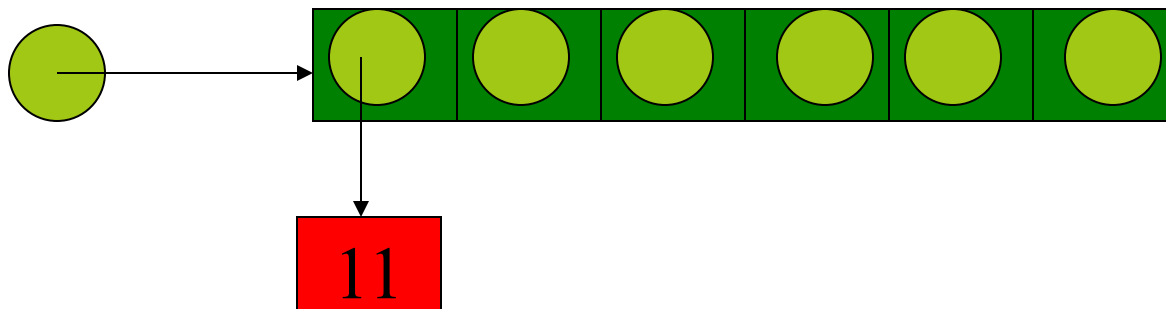
Array in Java

- Nel secondo caso invece, ogni elemento della array è solo un riferimento: se si vuole un nuovo oggetto bisogna crearlo

```
w = new Counter[6];
```



```
w[0] = new Counter(11);
```



Array in Java

- In quanto istanze di una classe “array”, gli array Java hanno alcune proprietà
- Tra queste, il campo dati pubblico `length` dà la lunghezza (dimensione) dell’array:

`v.length` vale 3

`w.length` vale 6

- Una volta creato, un array ha comunque dimensione fissa
 - non può essere “allargato” a piacere
 - per tali necessità esiste la classe `Vector`

ESERCIZIO

Stampare gli argomenti passati dalla linea di comando

```
public class EsempioMain{
    public static void main(String[] args){
        if (args.length == 0)
            System.out.println("Nessun argomento");
        else
            for (int i=0; i<args.length; i++)
                System.out.println("argomento " + i + ": " + args[i]);
    }
}
```

L'operatore **+** concatena **String** e anche valori di tipi primitivi (che vengono automaticamente convertiti in **String**).

In Java si possono definire variabili in ogni punto del programma (non solo all'inizio come in C): in particolare è possibile definire l'indice **i** dentro al ciclo **for**. Visibilità limitata al corpo del ciclo: fuori da esso, **i** risulta indefinita.

Riepilogo: Creazione di Array

- Dichiarazione – crea solo il riferimento - valore null

```
int myInts[];
```

```
int[] myInts;
```

- Istanziamento

```
myInts = new int[10];
```

- dichiarazione e istanziamento:

```
int[] myInts = new int[10];
```

- accesso a ciascuno elemento

```
myInts[3] = 9;
```

```
x = myInts[4];
```

- inizializzazione statica

```
int[] myInts = {1,2,5,6,7,4};
```

Array multidimensionali

- Gli Arrays possono contenere elementi che sono:
 - Primitivi
 - Oggetti
 - e Quindi Arrays! (ricorda l'array e' un oggetto)
- Cio ci permette di crea array 2-D, 3-D, 4-D, attraverso la definizione di arrays di arrays.
- Nel caso 2-D, noi avremo un array 1-D i cui elementi sono array 1-D.
- Avremo degli array 2-D piu' flessibili
 - Ogni riga dell'array non deve avere la stessa dimensione

Tipico uso del array 2-D

```
int [ ] [ ] multiplicationTable;  
  
multiplicationTable = new int [11] [21];  
  
for (int i=0; i < multiplicationTable.length; i++) {  
    for (int j=0; j < multiplicationTable[i].length; j++) {  
        multiplicationTable[i][j] = i*j;  
    }  
}
```

Dov'è l'errore?

Inizializzazione statica di array multidimensionali

Noi possiamo inizializzare un array multidimensionali nel seguente modo:

```
String s[][] =  
{  
    {"io", "sono", "la riga", "uno"},  
    {"io", "sono", "la", "seconda"},  
    {"io", "la", "terza"}  
};
```

s.length e' uguale a 3

S[0].length è uguale a 4

S[1].length è uguale a 4

S[2].length è uguale a 3

io	sono	la riga	uno
io	sono	la	seconda
io	la	terza	

```
public class ArrayDemo {  
    public static void main(String args[]) {  
        String s[][] = {  
            {"io", "sono", "la riga", "uno"},  
            {"io", "sono", "la", "seconda"},  
            {"io", "la", "terza"}  
        };  
        for(int row = 0; row < s.length; row++) {  
            System.out.println ("La riga " + row + " ha " + s[row].length + "  
colonne");  
            for(int col = 0; col < s[row].length; col++) {  
                System.out.print("<" + s[row][col] + "> ");  
            }  
            System.out.println();  
        }  
    }  
}
```

```
La riga 0 ha 4 colonne  
  <io>  <sono>  <la riga>  <uno>  
La riga 1 ha 4 colonne  
  <io>  <sono>  <la>  <seconda>  
La riga 2 ha 3 colonne  
  <io>  <la>  <terza>
```



```
public class Multiple{
    public static void main(String args[]){
        int[][] numbers;
        numbers = new int[5][];
        for(int row=0; row<numbers.length; row++){
            numbers[row] = new int[(int) (Math.random()*8)];
            for(int col=0; col<numbers[row].length; col++){
                numbers[row][col] = row*10 + col;
                if (row<1) System.out.print("0");
                System.out.print(numbers[row][col]+ " " );
            }
            System.out.println();
        }
    }
}
```

```
>java Multiple  
00 01 02  
10 11 12 13 14 15 16  
20 21 22 23  
30 31 32 33 34 35  
40 41 42
```

```
>java Multiple  
00 01  
10 11 12 13  
20 21 22 23 24 25 26  
30 31 32 33 34 35  
40 41 42
```

```
>java Multiple  
00 01 02 03  
10  
20  
30 31  
40
```

VECTOR

Vectors

Esempio: uso della classe Vector per creare una collezione di Counter

```
import java.util.*;
//Dichiarazione
Vector contatori= new Vector();
//Inserimento Elementi
contatori.addElement(c1);
contatori.addElement(c2);
//Accesso all'elemento i
c3 = (Counter) contatori.elementAt(i);
//Modifica di un elemento di posizione i
contatori.setElementAt(c4, i);
//Inserimento di un elemento in posizione i
contatatori.insertElementAt(c4, i);
//Rimozione un elemento di posizione I
contatori.removeElementAt(i);
```